

Compiler & Language Translators

1. Language Translators

A **language translator** converts a program from one language into another language.

Main types:

- **Compiler**
- **Interpreter**
- **Assembler**

Basic classification

Translator	Input	Output
Compiler	High-level language	Target/Object code
Interpreter	High-level language	Executes the program
Assembler	Assembly language	Machine/Object code

2. Compiler

A **compiler** translates a source program into an equivalent target program.

General flow

Source Program



Compiler



Target Program

The generated target code can be executed later.

Important points

- Compilation and execution are conceptually separate.
- Compiler can detect errors during compilation.
- Compiler may generate **intermediate code/representation** before producing target code.
- Optimization may be performed before final target-code generation.

3. Interpreter

An **interpreter** translates and executes a program during interpretation rather than producing a separate target executable in the traditional model.

Source Program



Interpreter



Translation + Execution

Compiler vs Interpreter

Compiler

Produces target code

Compilation occurs before execution

Usually faster execution after compilation

Errors can be reported during

Interpreter

Executes while interpreting

Interpretation and execution are coupled

Usually more interpretation overhead

Errors may appear as execution reaches the relevant

compilation

part

GATE Trap ⚠

Do **not** assume that every interpreter literally processes only one source line at a time. Modern language implementations may use **bytecode, JIT compilation, caching, etc.**

4. Assembler

An **assembler** converts an assembly-language program into machine/object code.

Assembly Language



Assembler



Object/Machine Code

Remember

Assembly → Machine/Object code

It does **not** normally translate a high-level language directly.

5. Complete Program Translation Process

A typical C/C++ toolchain can be represented as:

Source Program



Preprocessor



Compiler



Assembly Code



Assembler



Object Code



Linker



Executable



Loader



Main Memory



Execution

This sequence is **highly important for GATE**.

6. Preprocessor

The **preprocessor** performs preprocessing before actual compilation.

Common tasks include:

- Processing `#include`
- Macro expansion using `#define`
- Conditional compilation
- Handling preprocessing directives

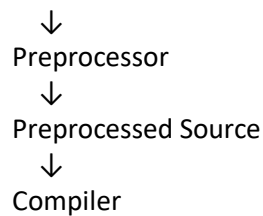
Example:

```
#include <stdio.h>
```

```
#define MAX 100
```

Conceptually:

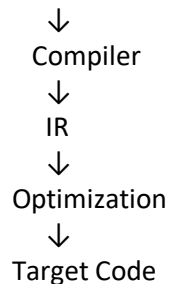
Source Code



7. Intermediate Representation (IR)

A compiler may translate source code into an **Intermediate Representation (IR)** before generating target code.

Source Code



Why IR is useful?

IR provides a common intermediate form between the source language and target machine.

It helps in:

- Compiler portability
- Code optimization
- Supporting multiple source/target combinations
- Separating front-end and back-end design

8. Compiler Front End and Back End

Front End

Mainly deals with the **source language**.

Typical activities:

Lexical Analysis

↓
Syntax Analysis

↓
Semantic Analysis

↓
Intermediate Representation

Back End

Mainly deals with the **target machine**.

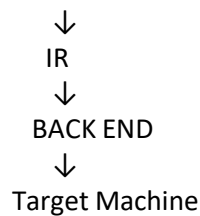
Typical activities:

IR
↓
Optimization
↓
Code Generation
↓
Target Code

GATE Concept ★

A common conceptual structure is:

Source Language
↓
FRONT END



9. Code Optimization

Optimization transforms a program into a **more efficient equivalent program** without changing its required behavior.

Possible goals:

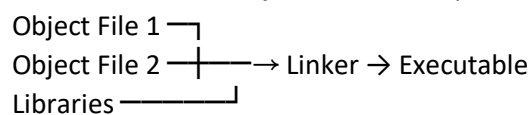
- Reduce execution time
- Reduce code size
- Reduce memory usage
- Improve resource utilization

Important

Optimization must preserve the **semantics/meaning** of the program.
The optimized code need not look identical to the original code.

10. Linker

A **linker** combines object files and required libraries/modules to produce an executable program.



Important distinction

Compiler: translates source program

Linker: combines object modules/libraries

Do not confuse them.

11. Static and Dynamic Linking

Static Linking

Required library code is incorporated into the executable during linking.

Object Code + Library Code



Advantage: fewer external library dependencies at execution time.

Disadvantage: executable can become larger.

Dynamic Linking

Some library code is linked/loaded separately, typically when required.

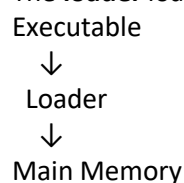
Examples of dynamic library formats:

- .dll — Windows
- .so — Linux/Unix

Advantage: libraries can be shared among multiple programs.

12. Loader

The **loader** loads the executable program into main memory so that it can be executed.





Execution

Remember the roles

Compiler → translates

Assembler → converts assembly to object/machine code

Linker → combines

Loader → loads

CPU → executes

★ GATE Quick Revision Table

Component	Main Job
Preprocessor	Processes directives/macros
Compiler	Source program → target/IR/assembly
IR	Intermediate form of program
Optimizer	Improves code while preserving meaning
Assembler	Assembly → object/machine code
Linker	Combines object files and libraries
Loader	Loads executable into memory
Interpreter	Interprets and executes program



GATE Must-Remember Facts

1. **Assembler works on assembly language.**
2. **Compiler and assembler are different.**
3. **Linker combines object modules and libraries.**
4. **Loader loads the executable into memory.**
5. A compiler **may use an intermediate representation.**
6. Optimization should **not change program semantics.**
7. Compiler and interpreter are **not necessarily mutually exclusive in modern implementations.**
8. **#include and #define are handled during preprocessing.**
9. **Static linking** incorporates library code into the executable.
10. **Dynamic linking** keeps some library code separate and links/loads it when required.



One-line memory trick

P → C → A → L → L

Preprocessor → Compiler → Assembler → Linker → Loader

Then:

CPU executes the loaded program.